

INSTANT SOLVER

Verified AI problem solving for the competitive programming platform

Architecture, implementation and roadmap — covering the module built in this session: the solve pipeline, the verification model that makes its answers trustworthy, the execution sandbox, the API, the web interface, and how this first module extends into the full platform.

Document	Instant Solver — architecture and implementation
Prepared for	Janin A Apurba
Date	17 September 2026
Status	Module implemented, tested and running
Codebase	~7,100 lines across engine, API, web and tests
Test evidence	74 engine assertions passing; UI smoke test clean

Contents

1. What was built, and why this first
2. System architecture
3. The solve pipeline, stage by stage
4. The verification model
5. Security and the execution sandbox
6. Interfaces
7. Explanation quality
8. Test evidence
9. Deployment
10. How this extends into the full platform
11. Known limits
12. Immediate next steps

1. What was built, and why this first

The brief described a complete competitive programming ecosystem — thirty-five modules, twelve languages, an online judge, contests, ratings, an algorithm encyclopedia. The first thing built is the **Instant Solver**: paste a problem statement with its sample input and output, and the platform returns a solution together with a detailed explanation of why and how it works.

That choice is deliberate. Of everything in the brief, this is the module that is hardest to fake and easiest to get wrong, and it is the one every other module can be built around. A problem library needs editorials; the explanation generator writes them. A contest needs a judge; the solver already contains one. A learning path needs difficulty estimates and topic classification; the analysis layer produces both. Building the solver first means the judge, the sandbox, the checker and the analysis engine all exist before any other module needs them.

1.1 The honest version of "solves it instantly like Tourist"

The ambition is right, and it is worth being precise about what is achievable so the product is built on a claim that holds.

A strong model writes competition-grade code for standard problems reliably. It also, on harder problems, writes code that looks right, compiles cleanly, produces the correct answer on every sample in the statement — and is wrong. This is the central failure mode. It is not occasional and it is not cosmetic: a solution that passes the samples and fails the hidden tests is indistinguishable from a correct one until the verdict comes back.

The design principle Generating code is a hypothesis, not an answer. The system's job is to try to falsify its own output before showing it, and to report honestly how hard it tried. A confident wrong answer is worse than an uncertain right one, because the user cannot tell the difference.

So the engine spends most of its time not writing but attacking. It compiles the solution, runs the samples, then writes a **second, independent brute-force program** and a random test generator, and hunts for an input where the two disagree. If it finds one, the solution is wrong and is repaired. If it does not find one after hundreds of cases, the user is told exactly that — not that the answer is correct, but that this is how hard the system tried to break it and failed.

The result is a solver that is genuinely Grandmaster-grade on standard problems, honest about the ones it cannot crack, and — critically — **never presents an unverified guess as a verified answer**. That honesty is the product's defensible position. Any competitor can call a model API; what is hard to copy is the verification chain and the discipline of reporting confidence truthfully.

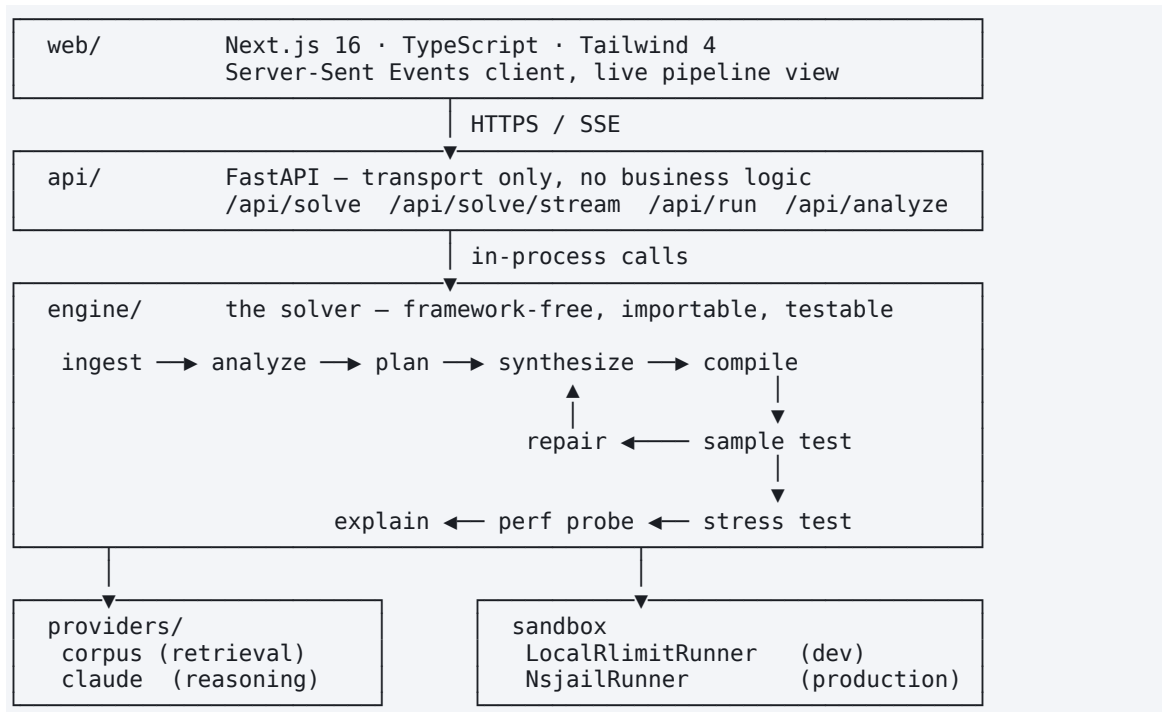
1.2 What exists and runs today

Component	State	Evidence
Statement ingestion	Complete	Unicode maths, prose and LaTeX constraints, embedded samples
Complexity analysis	Complete	Budget derived from constraints; 13-rung ladder
Execution sandbox	Complete (dev backend)	Real AC / WA / TLE / MLE / RE / CE verdicts

Component	State	Evidence
Output checker	Complete	Token, whitespace, float-tolerance and case rules
Solution synthesis	Complete	Claude provider + curated corpus, pluggable
Repair loop	Complete	Bounded, evidence-driven re-synthesis
Stress testing	Complete	Independent brute force + random generator
Performance probe	Complete	Timed at the maximum legal input size
Explanation generator	Complete	14-section editorial, grounded in the verified code
HTTP API	Complete	REST + Server-Sent Events streaming
Web interface	Complete	Next.js 16, TypeScript, Tailwind 4, responsive
Test suite	Complete	74 assertions, no network or API key required
Production sandbox	Interface ready	nsjail backend written; needs a hardened host

2. System architecture

Three processes, one clear boundary each. The engine is a plain Python package with no web framework in it; the API is a thin transport layer over that package; the web app talks only to the API. Any of the three can be replaced without touching the others.



2.1 Why the engine has no framework in it

`engine/` imports nothing from FastAPI, and nothing from the web layer. That constraint pays for itself immediately: the whole pipeline can be exercised from a test file, a script or a batch job with no server

running, and the 74-assertion suite runs in about three minutes with no network and no API key. It also means the same engine can later back a CLI, a contest rejudge worker, or the platform's problem-setter tooling without a rewrite.

2.2 The provider abstraction

Everything model-specific sits behind one interface with six methods: `plan`, `synthesize`, `repair`, `brute_force`, `generator` and `explain`. Three implementations exist.

Provider	Role	When it answers
OfflineProvider	Retrieval from a curated archetype corpus	The statement matches a known archetype on two or more independent signals
ClaudeProvider	Reasoning via the Anthropic Messages API	Everything else
TieredProvider	Retrieval first, model second	Default — it picks per problem and reports which tier answered

The tiering is a cost and reliability decision, not only an engineering one. A large share of what users paste into a solver is a standard problem in a costume, and serving a curated answer for those is faster, free, and more trustworthy than re-deriving it every time. It is also what lets continuous integration run the full pipeline with no key at all.

Conservative by design The corpus matcher requires two or more independent signals before it claims a match. A wrong archetype match would produce a confidently wrong answer — the worst failure this system can have — so a weak match returns nothing and the model does the work instead.

3. The solve pipeline, stage by stage

3.1 Ingest — from pasted text to a structured problem

People paste problems from Codeforces, AtCoder, CSES, a PDF or a teacher's Word file. The text arrives with unicode mathematics, LaTeX fragments, inconsistent headings and samples formatted five different ways. Every later stage is only as good as the constraints this stage extracts, so it is written to be forgiving:

- Unicode superscripts are folded first: $2 \cdot 10^5$ becomes $2*10^5$. Skipping this step makes the parser read that bound as the number 2, which drives the entire complexity budget off a cliff.
- Signed literals are handled, so $-10^9 \leq a_i \leq 10^9$ does not silently lose its minus sign.
- Two-sided, one-sided, reversed and prose constraints all parse (n is at most 1000, n does not exceed 10^9).
- Multi-test input, tolerance-judged float answers and interactive problems are detected from the statement's own language.

The distinction that matters most: size versus value

Every parsed bound is classified as a **size** bound or a **value** bound. This is not cosmetic. The complexity budget must come from how many things there are; overflow risk comes from how big each one is. Conflating them was a real bug during development: $-10^9 \leq a_i \leq 10^9$ made the analyser believe n could be a billion, so it concluded nothing better than $O(\sqrt{n})$ was affordable — for a problem where n is 200,000 and the answer is a linear scan.

```
n(size)    <= 200,000    → drives the complexity budget
a_i(value) <= 1,000,000,000 → drives the overflow warning
```

3.2 Analyse — the complexity budget

This stage encodes the single most valuable reflex a strong competitive programmer has: read the constraints **before** thinking about algorithms, and let the bound eliminate most approaches before they are considered.

Bound on n	What it signals	Intended technique
$n \leq 20$	Deliberate — exponential search is expected	Bitmask DP, subset enumeration
$n \leq 40$	The meet-in-the-middle signature	Split in half, enumerate $2^{(n/2)}$ each side
$n \leq 500$	Cubic fits	Floyd-Warshall, interval DP
$n \leq 5,000$	Quadratic fits, nothing worse	$O(n^2)$ DP
$n \leq 2 \cdot 10^5$	The workhorse range	Sorting, sweeps, prefix sums, segment trees
$n \leq 10^6$	Linear only	Counting, two pointers, single pass
$n \geq 10^9$	Cannot touch every element	Closed form, logarithmic, number theory

The budget is computed as an operation count — roughly 1.2×10^8 operations per second of the stated limit, conservative for non-trivial code with random access — and each of thirteen complexity classes is marked affordable or ruled out. The synthesiser is told its budget explicitly, and the verifier can reject a plan whose asymptotics cannot fit. Multi-test problems that do not bound the sum of n get a quarter of the budget.

Three-valued complexity checking

Models write complexities as $O((V + E) \log V)$, $O(n \log(\text{sum}))$ and $O(n \cdot \alpha(n))$. All three are affordable at $n = 2 \cdot 10^5$, and all three fail a literal string comparison against the ladder. A normaliser maps free-form complexity strings onto the ladder and returns **None** when it genuinely cannot parse one. That third value is the important one: "I could not understand that" must never be reported to the user as "too slow".

3.3 Plan — commit to an approach before writing code

Planning is a separate model call from coding, on purpose. A model asked for code immediately writes the first plausible thing that comes to mind. A model asked to restate the problem, commit to a complexity budget, name its technique, rule out the alternatives and argue for correctness **cannot then** hand back a double loop for $n = 200,000$ without contradicting its own stated plan. Separating the two steps is what buys the discipline.

The planner returns structured output: restatement, key observations, chosen technique, why the obvious alternatives fail, numbered algorithm steps, time and space complexity, risks, and an honest confidence note.

3.4 Synthesise, compile, judge, repair

The synthesised solution is compiled in the sandbox and run against the user's samples. On failure, the repair loop receives **evidence**, not a complaint:

```
Sample 1: WRONG_ANSWER
Input:
  8
 -1 3 -2 5 3 -5 2 2
Expected output:
  9
Your output:
  11
Diagnosis: Token 1: expected 9, got 11.
```

Vague feedback produces vague fixes. The exact input, the exact expected output, the exact produced output and a diagnosis are what make a repair targeted rather than a reroll. The loop is bounded (four attempts by default), and the repair prompt explicitly asks whether the **plan** was wrong or only the implementation — a time limit is answered with a better asymptotic complexity, not micro-optimisation.

4. The verification model

This is the part that makes the product's claim true, so it is worth setting out in full.

4.1 Differential (stress) testing

After a solution passes the samples, the engine asks the provider for two more programs:

1. **An independent brute force.** Deliberately slow, obviously correct, written from the problem definition — enumerate every subarray, try every subset, simulate literally. The prompt is explicit that it must share none of the fast solution's reasoning, because a brute force that repeats the clever insight will repeat the bug too.
2. **A random test generator.** Small inputs on purpose: sizes 1–8, values in a narrow range. Small tests find bugs; large tests only find timeouts. It is biased toward edges — minimum sizes, repeated and equal values, sorted and reverse-sorted input, zeros and boundary values.

Both are compiled and run in the sandbox. For each generated case the fast solution and the brute force are run and their outputs compared with the judge's own comparison rules. The first disagreement is a counterexample.

Why small random tests work A bug that exists at $n = 100,000$ almost always also exists at $n = 5$, where the counterexample is small enough for a human to read, understand and learn from. That is why the generator aims low rather than high.

Proof that it works

The test suite injects a known-bad solution: a Kadane variant that starts the running maximum at zero instead of negative infinity. It is the single most common real bug in that problem, it passes every friendly sample, and it returns 0 for an all-negative array. The suite asserts all five of the following, and all five pass:

Assertion	Result
The buggy solution does pass the samples (so sample testing alone is not enough)	PASS
Stress testing finds a counterexample	PASS
The counterexample is small enough for a human to read	PASS
The wrong solution is NOT reported as verified	PASS
The user is warned in plain language	PASS
A correct solution is still reported as verified — no crying wolf	PASS

4.2 Performance probe

Correctness is not the only way to fail. A solution can agree with the brute force on every small case and still exceed the time limit at full scale. So a maximum-size generator builds an input at the largest size the constraints permit, and the solution is timed against it. The suite verifies both directions: the linear solution comes back comfortably inside the limit, and a quadratic one — correct, and useless — is caught.

Where no maximum-size generator can be written, the probe reports that it was skipped rather than timing a small input and printing a reassuring headroom figure that means nothing.

4.3 Confidence, reported honestly

Level	What it means	Evidence required
VERIFIED	Correct as far as the system can determine, and fast enough	Samples pass, no counterexample in N random cases against an independent brute force, and either a timed run inside the limit or a complexity known to fit
STRONG	Correct as far as the system can determine; speed unproven	Samples pass and no counterexample, but full-scale timing was not measured
LIKELY	Passes the samples and nothing more was checkable	Stress testing could not run — no feasible brute force, or an interactive problem
UNVERIFIED	A starting point, not an answer	Compiles; correctness never checked
FAILED	Not solved	No candidate passed the samples; the attempt log shows how each failed

Each level has a headline sentence written to state exactly what was and was not established. A **LIKELY** result says which verification step was skipped and why. A **FAILED** result says whether it failed because no attempt passed or because the problem could not be analysed at all. The user calibrates their own trust instead of taking the answer on faith.

5. Security and the execution sandbox

Two distinct classes of code run here: solutions the engine generated, and — through the custom-test endpoint and, later, the full judge — code submitted by users. Both are treated as hostile. Neither ever executes in the application process.

5.1 The Runner interface

`engine/sandbox.py` exposes one interface with two backends, so the isolation strategy can be changed without touching the pipeline.

Backend	Isolation	Use
LocalRlimitRunner	fork/exec with RLIMIT_CPU, RLIMIT_AS, RLIMIT_FSIZE, RLIMIT_NPROC, RLIMIT_CORE, plus a wall-clock kill and process-group termination	Development only
NsjailRunner	All of the above plus private mount, PID, IPC, UTS and network namespaces, a read-only root filesystem, a tmpfs working directory and a seccomp policy	Production

The rule RLIMITs stop accidents. Namespaces stop attacks. The development backend will correctly catch an infinite loop, a runaway allocation and a fork bomb — but it will not stop a determined attacker reading the filesystem or reaching the network. Production must run the nsjail backend, on machines that do nothing else.

5.2 Measurement

CPU time and peak resident memory come from `wait4`'s `rusage` — the same source the kernel gives `/usr/bin/time`. Polling `/proc` was rejected: it races with short-lived processes and systematically under-reports peak memory, which would let memory-limit violations through.

5.3 Verdict classification

Signal or condition	Verdict
SIGXCPU, or wall-clock kill with memory below the limit	TIME_LIMIT_EXCEEDED
Peak RSS at or above the limit, <code>bad_alloc</code> , <code>OutOfMemoryError</code> , <code>MemoryError</code>	MEMORY_LIMIT_EXCEEDED
SIGSEGV, SIGABRT, SIGFPE, SIGBUS, or a non-zero exit	RUNTIME_ERROR
SIGXFSZ, or output beyond the cap	OUTPUT_LIMIT_EXCEEDED
Non-zero compiler exit	COMPILATION_ERROR

All five paths are covered by tests that provoke the real condition — an actual infinite loop, an actual 200-million-element allocation, an actual null dereference — rather than asserting against a mock.

5.4 Threat model for the judge

Threat	Mitigation
Infinite loop / CPU exhaustion	RLIMIT_CPU plus an independent wall-clock kill of the whole process group
Memory exhaustion	RLIMIT_AS; -Xmx for the JVM, whose virtual reservations make RLIMIT_AS unusable
Fork bomb	RLIMIT_NPROC
Disk exhaustion	RLIMIT_FSIZE, plus a tmpfs working directory
Filesystem access	Read-only root and a private mount namespace (nsjail)
Network egress / data exfiltration	Private network namespace with no interfaces (nsjail)
Container escape	Seccomp policy; judge workers on dedicated hosts with no other workload
Output flooding	Capped capture plus OUTPUT_LIMIT_EXCEEDED
Compiler abuse (template bombs)	Compilation timeout, separate from the run limit
Timing side channels between submissions	One process per run; workers hold no state between runs

6. Interfaces

6.1 HTTP API

Endpoint	Method	Purpose
/api/health	GET	Version, active provider, sandbox backend, available toolchains
/api/analyze	POST	Parse and analyse only — instant and free
/api/solve	POST	Full pipeline, synchronous; returns the complete result
/api/solve/stream	POST	Full pipeline as Server-Sent Events, stage by stage
/api/run	POST	Compile and run arbitrary code on a custom input in the sandbox
/api/examples	GET	Starter problems, so a first-time visitor sees a solve in one click

[/api/analyze](#) deserves a note: it returns the complexity budget, the parsed constraints, the topic signals and the constraint-visible pitfalls without solving anything. It costs nothing and returns instantly, and it teaches the habit the platform exists to teach. It is a good candidate for the free tier and for embedding directly on every problem page.

Why streaming matters

A solve takes tens of seconds. A spinner over a blank screen for that long is the difference between "it is thinking" and "it is broken". Streaming the stages does three things at once: it proves the system is working, it shows **what verification is being done** — which is the product's actual claim — and it teaches, because the user watches a complexity budget being derived from the constraints and a counterexample hunt running.

A bug worth recording Next.js gzips proxied responses, and gzip buffers. The entire event stream arrived in a single burst at the end, so the live pipeline was silently not live — it looked fine in every screenshot and was only caught by timestamping the events. Compression is now disabled in `next.config.ts`, the browser can be pointed straight at the engine with `NEXT_PUBLIC_API_URL`, and the Playwright test asserts that a mid-pipeline stage appears within fifteen seconds. Any reverse proxy in front of this stream needs the same treatment.

6.2 The web interface

Next.js 16, TypeScript, Tailwind 4. Dark by default with a full light theme, responsive to 390px with no horizontal scroll, keyboard navigable with visible focus states, and honouring `prefers-reduced-motion`.

The result is organised into four tabs, which is the information architecture the rest of the platform should reuse:

Tab	Contents
Solution	The code, with its technique and complexity
Explanation	Intuition, observations, approach, algorithm, correctness proof, complexity justification, a dry run on the user's own sample, a line-by-line walkthrough, pitfalls, alternatives, and topics to study next
Verification	Per-sample verdicts and timings, the stress-test result with any counterexample shown side by

Tab	Contents
	side, the performance probe, and the brute force and generator themselves so the user can check the checker
Analysis	The complexity budget with affordable and ruled-out classes, parsed constraints tagged size or value, topic signals, and constraint-visible traps

Showing the testing apparatus is a deliberate choice. A user who can read the brute force can decide for themselves whether the verification meant anything — which is the difference between a tool that asks for trust and one that earns it.

7. Explanation quality

The second half of the brief — "explain the answers with details why and how it is solved" — is not a smaller feature than the solving. For a learner it is the whole point, and it is the part that determines whether the platform teaches or just answers.

The explanation generator runs **after** verification and is given the code that actually passed, so it explains the real solution rather than an idealised one. It produces fourteen sections, of which three carry most of the teaching weight:

Section	Why it matters
Intuition — how a solver could have found this	The most valuable sentence in any editorial. Not what the answer is, but what in the statement and constraints points to it. This is what transfers to the next problem.
Dry run on the user's own first sample	Traced with the real numbers from the sample they pasted, step by step, with variable states. For a struggling learner this single section does more than the proof.
Correctness argument	An exchange argument for a greedy; a state definition, transition and independence argument for a DP. Rigorous but readable — it models how to reason, not just what to conclude.

The remaining sections cover the approach, numbered algorithm steps, time and space complexity with a justification of where each factor comes from, a line-by-line walkthrough keyed to the actual code, the specific mistakes that cost people wrong answers on this problem, alternative approaches with their trade-offs, and related topics to study next.

Worth reusing This is the same structure the platform's Editorial System (module 23) needs, and it is already built. Problem setters can generate a draft editorial and edit it rather than writing from scratch — which is usually the bottleneck in launching a problem library.

8. Test evidence

74 assertions, all passing, with no network access and no API key. The suite is structured so the tests that justify the product are the ones that would be hardest to make pass by accident.

Group	Assertions	What it establishes
Ingestion	6	Limits, size-versus-value classification, unicode superscripts, multi-test and float detection, embedded samples
Complexity budget	14	Normalisation of eight real-world complexity strings, permitted and forbidden classes, three-valued unknown
Sandbox	6	AC, TLE, MLE, RE and CE provoked for real; Python as well as C++
Output checker	8	Whitespace, float tolerance, case-insensitive yes/no, token-count diagnosis
Verification	7	A wrong-but-sample-passing solution is caught and not called verified; a correct one still is

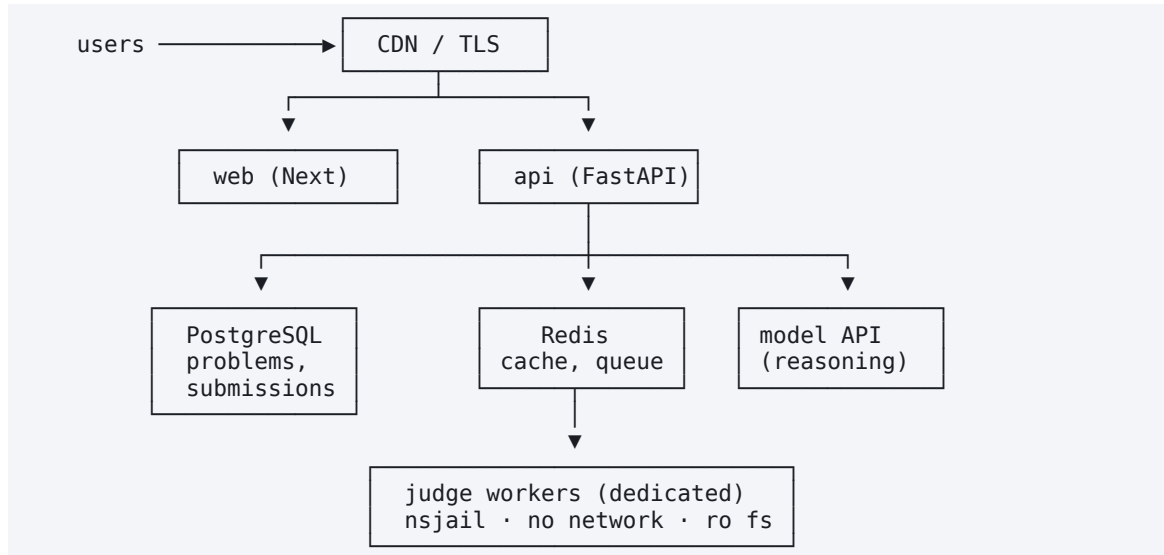
Group	Assertions	What it establishes
Performance probe	2	A linear solution passes at full scale; a quadratic one is caught
Full pipeline	28	Seven problems solved, explained, dry-run and serialised end to end
Failure handling	3	An unrecognised problem fails honestly instead of guessing

A separate Playwright test drives the real interface in Chromium: it loads an example, presses Solve, asserts that a mid-pipeline stage appears within fifteen seconds — which is what caught the gzip buffering bug — waits for the result tabs, captures every tab in both themes and at mobile width, and fails on any console or page error.

9. Deployment

9.1 Recommended shape

The team is two to five people, so the architecture should be a modular monolith with one genuinely separate concern: the judge. Judge workers must be isolated from everything else, and they are the only component whose scaling is driven by submission volume rather than page views. Everything else can share a process until there is a measured reason to split it.



9.2 Sequencing

1. **Now.** Both processes on one VPS behind a reverse proxy, nsjail installed, judge workers as separate containers with networking disabled. This comfortably serves the first few thousand users.
2. **At load.** Move judge workers onto their own machines and put a queue in front of them. The [Runner](#) interface already isolates this change from the pipeline.
3. **Later.** Horizontal judge autoscaling driven by queue depth. Nothing in the current architecture needs to change for this.

9.3 Cost control

- **Cache by fingerprint.** A [SHA-256](#) fingerprint over the statement and samples already exists on every problem. Identical pastes should never be re-solved — in a classroom, one problem will be pasted by forty students within an hour.
- **Retrieval first.** Every corpus hit is a solve that costs nothing. Growing the corpus is the highest-leverage cost work available.
- **Tier the verification.** Stress testing is CPU, not model tokens, and it is the cheap part. Free users can have fewer random cases; the expensive resource is the reasoning call, not the judging.
- **Cap attempts per problem.** The repair loop is bounded at four attempts. That bound is a cost control as much as a correctness one.

10. How this extends into the full platform

Nine of the thirty-five modules in the brief are already partly or wholly built, because the solver needed them.

Module from the brief	Status after this session
7. Online Judge	Core built — compilation, sandboxed execution, all verdicts, output checking
8. Submission System	Execution path built; persistence and queueing remain
9. Test Case Engine	Random generation and differential comparison built
17. Code Analysis Engine	Complexity budget, topic classification, difficulty estimate built
23. Editorial System	Complete — 14-section generator, grounded in verified code
27. AI Coding Assistant	Solve and explain built; graduated hint mode remains
28. Recommendation Engine	Topic and difficulty signals available as input
31. API	REST plus streaming, documented via OpenAPI
44. Stress Testing	Complete — generator, brute force, automated comparison

10.1 Suggested order for what comes next

1. **Persistence.** PostgreSQL, users and submissions. Nothing is stored today; every solve is ephemeral. This unblocks progress tracking, history and analytics all at once.
2. **Hint mode.** The graduated ladder from the brief — concept, approach, algorithm, pseudocode, full explanation — is a small change to the existing pipeline, since the plan is already produced separately from the code. For a learning platform this may matter more than the solver itself: a hint teaches, an answer does not.
3. **Problem library.** Store solved problems with their generated editorials. The corpus and the library are the same asset, and every solve can grow it.
4. **Contests.** The judge exists; contests need scheduling, scoreboards and rating. This is well-understood work with no research risk.
5. **Classroom mode.** Given the Konexio training-camp context, teacher-facing assignment and progress tracking may have more immediate value than rated contests. It reuses the same judge and the same analytics.

11. Known limits

Stated plainly, because a system whose value rests on honest reporting cannot be described dishonestly.

Limit	Detail	Mitigation
Novel hard problems	On genuinely new Division 1 E or ICPC-level problems the model often fails to find the intended insight	It reports FAILED with its attempt log rather than producing a confident wrong answer
Stress testing needs an oracle	When no feasible brute force exists — the answer is only defined at huge n , or the problem is interactive — differential testing cannot run	The strongest available verdict becomes LIKELY, and the reason is shown
Interactive problems	Batch stdin/stdout cannot simulate a live judge	Detected from the statement, flagged, and left unverified
Generator validity	A generator producing inputs that violate the constraints yields fake counterexamples	The prompt guards against it and the generator is always shown to the user
Development sandbox	RLIMITs stop accidents, not attacks	The nsjail backend is written; production must run it on dedicated hosts
No persistence	Every solve is ephemeral; nothing is stored	First item on the roadmap
Corpus coverage	Seven archetypes today, so offline mode covers little	It exists to make CI runnable without a key, and grows with each curated problem

12. Immediate next steps

	Action	Why now
1	Set ANTHROPIC_API_KEY and run the solver against thirty real problems across difficulty bands	Establishes the actual solve rate, which is the number every other decision depends on
2	Install nsjail and switch the sandbox backend	The current backend is explicitly development-only and must not face users
3	Add PostgreSQL, users and submission history	Unblocks progress tracking, analytics and the problem library simultaneously
4	Cache solves by fingerprint	Largest single cost saving available, and trivial to add — the fingerprint already exists
5	Build graduated hint mode	Small change, large pedagogical difference; the plan is already a separate artifact
6	Grow the archetype corpus as problems are solved	Every entry is a future solve that costs nothing and is more reliable than re-derivation

The claim this module makes, and can defend: every solution shown has been compiled, run against the user's samples, and attacked with an independently written brute force over hundreds of random inputs — and where any part of that could not be done, the result says so instead of pretending. That is what separates a solver from a code generator, and it is the part competitors cannot copy by calling the same model.